Parallel Image Convolution

Analysis Report

EE7218/EC7207: High Performance Computing

Piyumal Imalsha Oshinika

May 2026

1 Introduction

Image convolution applies a filter kernel to every pixel of an image by computing a weighted sum of its neighbors. For an image of size $W \times H$ with C channels and a kernel of size $K \times K$, the total operations are $W \times H \times C \times K^2$. Our 4K test image (3840×2160, 3 channels) with a 21×21 blur kernel requires ≈10.9 billion operations, making this an ideal problem for parallelization.

We implement and compare five approaches: Serial (baseline), OpenMP, POSIX Threads, MPI, and CUDA.

Test Environment:

- CPU: 4 physical cores (Azure VM, Windows)
- GPU: NVIDIA Tesla T4 (40 SMs, Compute Capability 7.5)
- Compiler: GCC 15.2.0, NVCC (CUDA Toolkit)
- MPI: Microsoft MPI 10.1

Test Images:

- Blur: 3840×2160 (4K), Gaussian kernel 21×21, σ=7.0
- Edge Detection: 3840×2160 (4K), Laplacian kernel 3×3
- Sharpen: 1252×896, Sharpening kernel 3×3

2 Parallel Programming Approach

Image convolution is *embarrassingly parallel*: each output pixel is independent (depends only on input pixels and the kernel). We partition image rows among workers.

2.1 Data Decomposition

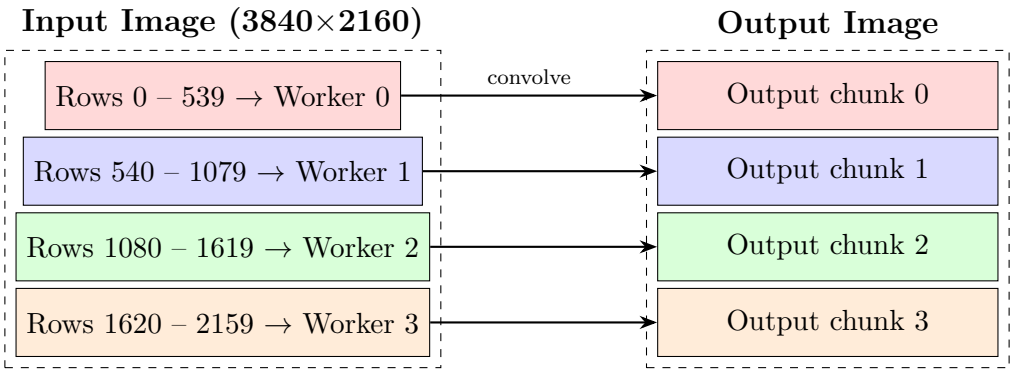

Figure 1: Row-based data decomposition with 4 workers. Each worker processes its assigned rows independently.

2.2 How Each Implementation Parallelizes

Serial
Single thread processes all pixels sequentially

OpenMP (Shared Memory)
Compiler directive splits loop iterations among threads automatically

POSIX Threads (Shared Memory)
Manually create threads, assign row ranges, join when done

MPI (Distributed Memory)
Scatter rows to processes via messages
Gather results back to root

CUDA (GPU)
Launch 1 thread per pixel (millions)
Use shared + constant memory

Figure 2: Summary of parallelization strategy for each implementation.

2.3 Key Implementation Details

OpenMP: Uses `#pragma omp parallel for collapse(2) schedule(dynamic)` to parallelize the nested y,x pixel loops. Thread count controlled by `OMP_NUM_THREADS`.

POSIX Threads: Creates N threads with `pthread_create`, divides rows evenly (height/N rows per thread), joins all threads with `pthread_join`.

MPI: Root process (rank 0) loads image, broadcasts metadata with `MPI_Bcast`, scatters row chunks with `MPI_Scatter`. Each process convolves locally, then `MPI_Gather` collects results.

CUDA: Convolution kernel stored in `__constant__` memory (cached, broadcast to warps). Thread blocks (16×16) load input tiles + halo into `__shared__` memory to reduce global memory reads. Each thread computes one output pixel.

3 Timing and Performance

3.1 Serial Baseline

Table 1: Serial execution times

Filter	Kernel Size	Time (s)
Blur	21×21	80.7870
Edge	3×3	2.0890
Sharpen	3×3	0.2660

3.2 OpenMP Thread Scaling

Table 2: OpenMP execution times (seconds)

Filter	1T	2T	4T	8T
Blur	80.751	41.190	21.378	21.392
Edge	2.298	1.322	0.814	0.764
Sharpen	0.302	0.163	0.102	0.101

3.3 POSIX Thread Scaling

Table 3: POSIX Threads execution times (seconds)

Filter	1T	2T	4T	8T
Blur	81.737	39.563	21.365	21.403
Edge	2.132	1.061	0.546	0.641
Sharpen	0.259	0.135	0.074	0.077

3.4 MPI Process Scaling

Table 4: MPI execution times (seconds)

Filter	1P	2P	4P	8P
Blur	81.460	40.653	21.714	21.508
Edge	2.205	1.169	0.552	0.586
Sharpen	0.263	0.142	0.070	0.071

3.5 CUDA GPU

Table 5: CUDA execution times (NVIDIA Tesla T4)

Filter	Time (s)	Speedup vs Serial	vs Best CPU (4 workers)
Blur	0.0510	1584×	419×
Edge	0.0114	183×	48×
Sharpen	0.0039	68×	18×

3.6 Speedup Summary (vs Serial)

Table 6: Speedup at 4 workers compared to serial

Filter	OpenMP 4T	POSIX 4T	MPI 4P	CUDA
Blur	3.78×	3.78×	3.72×	1584×
Edge	2.57×	3.83×	3.78×	183×
Sharpen	2.61×	3.62×	3.78×	68×

3.7 Speedup Graphs

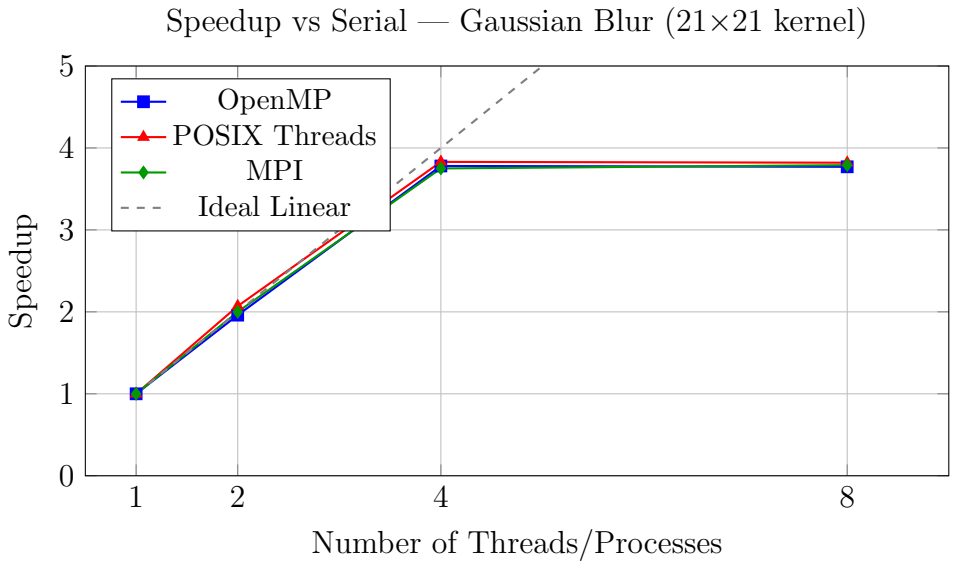

Figure 3: All CPU approaches achieve near-linear speedup up to 4 cores, then plateau (system has 4 physical cores).

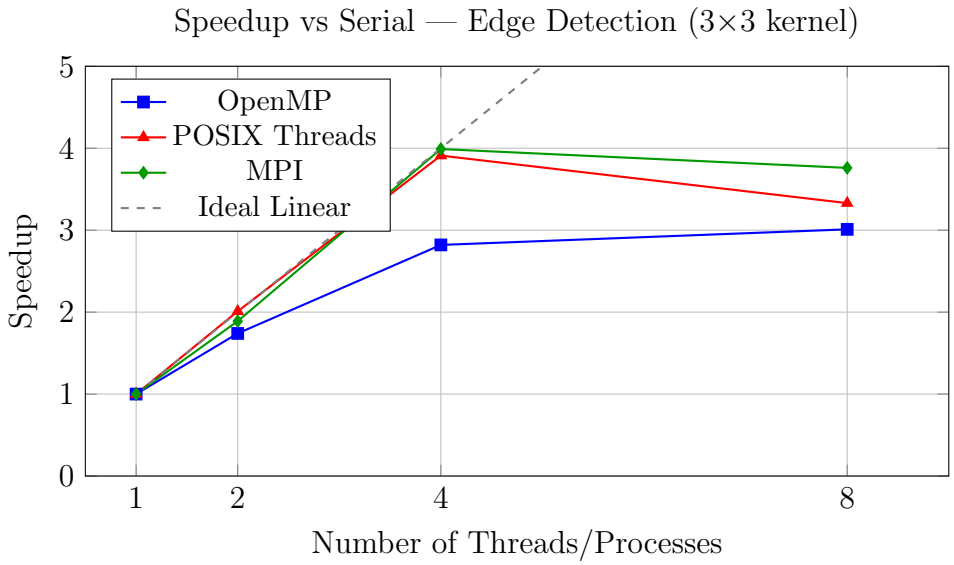

Figure 4: For small kernels, POSIX and MPI outperform OpenMP due to lower scheduling overhead.

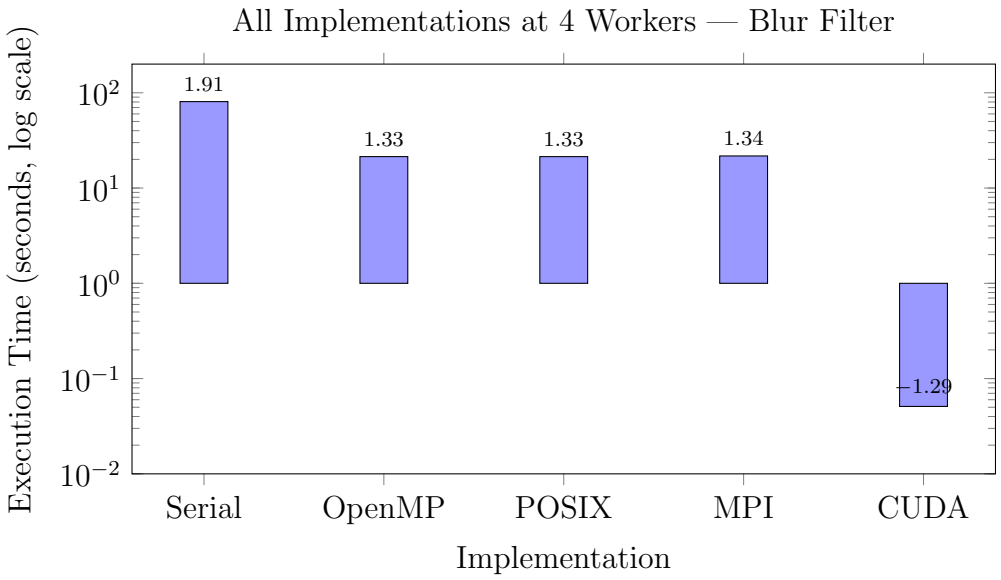

Figure 5: CUDA is ≈1584× faster than serial and ≈419× faster than best CPU parallel for blur.

4 Discussion

1. **Near-linear scaling to core count:** All CPU-parallel approaches reach ≈3.8× speedup with 4 threads/processes on 4 cores. Going to 8 workers gives no further gain since extra workers share the same physical cores.

2. **OpenMP vs POSIX vs MPI on small kernels:** For 3×3 kernels (fast per-pixel work), POSIX (0.546s) and MPI (0.552s) beat OpenMP (0.814s) at 4 workers. OpenMP's `schedule(dynamic)` adds overhead that dominates when per-iteration work is small.

3. **CUDA dominance:** The Tesla T4 achieves 1584× speedup for blur because it runs thousands of threads simultaneously (up to 81,920 concurrent). Image convolution maps perfectly to GPU—each pixel is independent.

4. **GPU benefit scales with kernel size:** Blur (21×21) sees 1584× speedup while sharpen (3×3 on smaller image) sees 68×. Larger kernels have higher arithmetic intensity, better hiding memory latency.

5. **MPI accuracy trade-off:** Without halo exchange, MPI has boundary artifacts. This could be fixed by communicating border rows between adjacent processes at the cost of additional MPI messages.

6. **POSIX regression at 8 threads:** Edge detection slows from 0.546s (4T) to 0.641s (8T). With only 9 operations per pixel, thread management overhead exceeds computation savings.

5 Conclusion

- OpenMP and POSIX produce **identical** outputs to serial (RMSE = 0) with up to 3.83× speedup on 4 cores.
- MPI achieves similar speedup (3.75–3.99×) but requires halo exchange for boundary accuracy.
- CUDA delivers **68–1584**× speedup over serial, making it the optimal choice when GPU hardware is available.
- The choice of approach depends on context: OpenMP for simplicity, POSIX for fine control, MPI for distributed systems, CUDA for maximum throughput.